

Lesson:

Methods of DOM – Part 2



Topics Covered

1. innerText
2. innerHTML
3. textContent
4. setAttribute()
5. getAttribute()

innerText

The **innerText** property is a DOM property in JavaScript that allows you to get or set the text content of an element, without including any HTML markup that may be present. This property can be used to get the text content of an element, which includes all the text nodes that are direct children of the element, or to set the text content of an element, which can be used to update the text of an element dynamically.

The syntax for using the **innerText** property to get the text content of an element is:

```
const elementText = element.innerText;
```

In this syntax, **element** is a reference to the element whose text content you want to retrieve, and **elementText** is a string that contains the text content of the element, without any HTML markup.

Here is an example that demonstrates how to use the **innerText** property to retrieve the text content of a **div** element:

```
<div id="my-div">
  Some text inside the div
  <p>Some more text inside the div</p>
</div>

<script>
const myDiv = document.querySelector('#my-div');
const myDivText = myDiv.innerText;
console.log(myDivText);
</script>
```

In this example, we use the **querySelector()** method to select an existing **div** element with an ID of **my-div**. We then use the **innerText** property to retrieve the text content of the **myDiv** element and log it to the console. The resulting output would be:

```
Some text inside the div  
Some more text inside the div
```

As you can see, the **innerText** property returns the text content of the **myDiv** element, without any HTML markup that may be present.

The syntax for using the **innerText** property to set the text content of an element is:

```
element.innerText = newText;
```

In this syntax, **element** is a reference to the element whose text content you want to set, and **newText** is a string that contains the new text content that you want to set for the element.

Here is an example that demonstrates how to use the **innerText** property to set the text content of a **div** element:

```
<div id="my-div">  
  Some text inside the div  
  <p>Some more text inside the div</p>  
</div>  
  
<script>  
const myDiv = document.querySelector('#my-div');  
myDiv.innerText = 'New text inside the div';  
</script>
```

In this example, we use the **querySelector()** method to select an existing div element with an ID of **my-div**. We then use the **innerText** property to set the new text content of the myDiv element to **New text inside the div**. The resulting HTML would look like this:

```
<div id="my-div">  
  New text inside the div  
  <p>Some more text inside the div</p>  
</div>
```

As you can see, the **innerText** property replaces the existing text content of the myDiv element with the new text content specified in the newText variable, without affecting any of the child elements of the element.

Note that the **innerText** property is similar to the **textContent** property, which also allows you to get or set the text content of an element. However, the textContent property includes all text nodes, including whitespace, while the innerText property removes leading and trailing whitespace and collapses consecutive whitespace characters. Additionally, the **innerText** property is not supported in older versions of Internet Explorer, while the **textContent** property is.

innerHTML

The **innerHTML** property in JavaScript is a DOM property that allows you to get or set the HTML content of an element. This property can be used to get the inner HTML of an element, which includes all the child elements and their contents or to set the inner HTML of an element, which can be used to add or remove child elements and their contents dynamically.

The syntax for using the **innerHTML** property to get the HTML content of an element is

```
const elementHTML = element.innerHTML;
```

In this syntax, **element** is a reference to the element whose HTML content you want to retrieve, and **elementHTML** is a string that contains the HTML content of the element, including all its child elements.

Here is an example that demonstrates how to use the **innerHTML** property to retrieve the HTML content of a **div** element:

```
<div id="my-div">
  <p>Some text inside the div</p>
  <ul>
    <li>List item 1</li>
    <li>List item 2</li>
    <li>List item 3</li>
  </ul>
</div>

<script>
const myDiv = document.querySelector('#my-div');
const myDivHTML = myDiv.innerHTML;
console.log(myDivHTML);
</script>
```

In this example, we use the **querySelector()** method to select an existing div element with an ID of **my-div**. We then use the **innerHTML** property to retrieve the HTML content of the **myDiv** element and log it to the console. The resulting output would be:

```
<p>Some text inside the div</p>
<ul>
  <li>List item 1</li>
  <li>List item 2</li>
  <li>List item 3</li>
</ul>
```

As you can see, the **innerHTML** property returns the full HTML content of the **myDiv** element, including all its child elements.

The syntax for using the **innerHTML** property to set the HTML content of an element is:

```
element.innerHTML = newHTML;
```

In this syntax, **element** is a reference to the element whose HTML content you want to set, and **newHTML** is a string that contains the new HTML content that you want to set for the element.

Here is an example that demonstrates how to use the **innerHTML** property to set the HTML content of a **div** element:

```
<div id="my-div">
  <p>Some text inside the div</p>
</div>

<script>
const myDiv = document.querySelector('#my-div');
const newHTML = '<p>New text inside the div</p>';
myDiv.innerHTML = newHTML;
</script>
```

In this example, we use the **querySelector()** method to select an existing **div** element with an ID of **my-div**. We then use the **innerHTML** property to set the new HTML content of the **myDiv** element to **<p>New text inside the div</p>**. The resulting HTML would look like this:

```
<div id="my-div">
  <p>New text inside the div</p>
</div>
```

As you can see, the **innerHTML** property replaces the existing HTML content of the myDiv element with the new HTML content specified in the **newHTML** variable.

Note that the **innerHTML** property can be used to add or remove child elements and their contents dynamically, but it can also be a security risk if you are not careful with the content that you insert into the page. You should never use the **innerHTML** property to insert untrusted content into the page, as this can open up security vulnerabilities such as cross-site scripting.

textContent

The **textContent** property is a DOM property in JavaScript that allows you to get or set the text content of an element, including all text nodes that are direct and indirect children of the element. Unlike the **innerText** property, it returns all the text nodes, including whitespace, and doesn't collapse whitespace characters.

The syntax for using the **textContent** property to get the text content of an element is:

```
const elementText = element.textContent;
```

In this syntax, **element** is a reference to the element whose text content you want to retrieve, and **elementText** is a string that contains the text content of the element, including any whitespace and text nodes.

Here is an example that demonstrates how to use the **textContent** property to retrieve the text content of a **div** element:

```
<div id="my-div">
  Some text inside the div
  <p>Some more text inside the div</p>
</div>

<script>
const myDiv = document.querySelector('#my-div');
const myDivText = myDiv.textContent;
console.log(myDivText);
</script>
```

In this example, we use the **querySelector()** method to select an existing **div** element with an ID of **my-div**. We then use the **textContent** property to retrieve the text content of the **myDiv** element and log it to the console. The resulting output would be:

```
Some text inside the div
Some more text inside the div
```

As you can see, the **textContent** property returns the text content of the **myDiv** element, including any whitespace and text nodes.

The syntax for using the **textContent** property to set the text content of an element is:

```
element.textContent = newText;
```

In this syntax, **element** is a reference to the element whose text content you want to set, and **newText** is a string that contains the new text content that you want to set for the element.

Here is an example that demonstrates how to use the **textContent** property to set the text content of a **div** element:

```
<div id="my-div">
  Some text inside the div
  <p>Some more text inside the div</p>
</div>

<script>
const myDiv = document.querySelector('#my-div');
myDiv.textContent = 'New text inside the div';
</script>
```

In this example, we use the **querySelector()** method to select an existing **div** element with an ID of **my-div**. We then use the **textContent** property to set the new text content of the **myDiv** element to **New text inside the div**. The resulting HTML would look like this:

```
<div id="my-div">
  New text inside the div
  <p>Some more text inside the div</p>
</div>
```

As you can see, the **textContent** property replaces the existing text content of the **myDiv** element with the new text content specified in the **newText** variable, including any whitespace and text nodes.

In summary, the **textContent** property returns all the text nodes of an element, including any whitespace, and allows you to set the text content of an element with the new text content including any whitespace. It is useful when you need to get or set the entire text content of an element including whitespace and text nodes.

setAttribute()

The **setAttribute()** method is a core feature of the Document Object Model (DOM) in JavaScript that empowers you to manipulate and modify attributes of HTML elements dynamically. It provides a flexible way to enhance the behavior, appearance, and functionality of web pages based on user interactions, data changes, or programmatic logic.

Syntax:

```
element.setAttribute(name, value);
```

- **element:** A reference to the HTML element on which you want to modify or add an attribute.
- **name:** The name of the attribute you want to modify or add, provided as a string.
- **value:** The value you intend to assign to the attribute, represented as a string.

Functionalities of setAttribute():

- **Modify Existing Attributes:**

You can use `setAttribute()` to alter attributes that are already present on an element. This is especially useful for updating attributes that affect an element's behavior or appearance based on changing conditions.

```
const button = document.getElementById('myButton');  
button.setAttribute('disabled', 'true'); // Disables the  
button
```

- **Add Custom Attributes:**

`setAttribute()` allows you to introduce custom attributes to elements. These attributes can store additional information that might be necessary for JavaScript operations or other purposes.

```
const input = document.querySelector('input');
input.setAttribute('data-validation', 'required'); // Adds a
custom 'data-validation' attribute
```

- **Dynamic Styling:**

By manipulating attributes like `class` or `style`, you can dynamically apply CSS styles to elements, making them responsive to user interactions.

```
const element = document.getElementById('myElement');
element.setAttribute('class', 'highlighted'); // Applies the
'highlighted' CSS class
```

- **Updating Links:**

The `href` attribute of anchor (`<a>`) elements can be modified to update navigation links, providing a seamless user experience in single-page applications.

```
const link = document.getElementById('myLink');
link.setAttribute('href', '/new-page'); // Updates the link
destination
```

- **Interactive Elements:**

Control the state of interactive elements, such as checkboxes or radio buttons, by modifying attributes like `checked` or `disabled`.

```
const checkbox = document.getElementById('myCheckbox');
checkbox.setAttribute('checked', 'true'); // Checks the
checkbox
```

- **Data Management:**

Custom attributes prefixed with data- can store valuable data for elements, helping in data exchange between JavaScript and HTML.

```
const element = document.getElementById('myElement');
element.setAttribute('data-id', '123'); // Adds a custom data attribute
```

When using these DOM method, it's important to consider the following points:

- **Security and Sanitization:**

Ensure that the values you assign to attributes are properly sanitized and validated. Prevent potential security risks, such as cross-site scripting (XSS), by carefully handling user inputs and avoiding direct insertion of untrusted content into attributes.

- **Attribute Case Sensitivity:**

Attribute names are case-insensitive in HTML but case-sensitive in XHTML. Maintain consistent casing when using `setAttribute()` to avoid any unexpected behavior.

- **Browser Compatibility:**

While `setAttribute()` is supported across modern browsers, be aware of potential differences in behavior, especially in older versions of Internet Explorer.

In conclusion, the `setAttribute()` method is a versatile tool for transforming the attributes of HTML elements within the DOM. It empowers you to create dynamic, interactive, and responsive web applications by modifying and adding attributes based on user interactions and programmatic requirements.

getAttribute()

The **getAttribute()** method, an integral component of the Document Object Model (DOM) in JavaScript, grants you the ability to retrieve attribute values from HTML elements dynamically. It serves as a gateway to access and utilize attribute data, enabling various actions based on these values.

- **Syntax:**

```
const attributeValue = element.getAttribute(name);
```

- **element:** A reference to the HTML element from which you wish to retrieve the attribute value.
- **name:** The name of the attribute you aim to extract (provided as a string).
- **attributeValue:** The value of the specified attribute, or null if the attribute isn't present.

Functionalities of `getAttribute()`

- **Retrieve Attribute Values:**

Utilize `getAttribute()` to extract attribute values, whether they're standard HTML attributes or custom data attributes (prefixed with `data-`). This is particularly handy for processing and making decisions based on these values.

```
const link = document.querySelector('a');
const hrefValue = link.getAttribute('href'); // Retrieves the
'href' attribute value
```

- **Conditional Logic:**

Use the retrieved attribute values for conditional statements, enhancing your application's responsiveness and interactivity.

```
const checkbox = document.getElementById('myCheckbox');
const isChecked = checkbox.getAttribute('checked');
if (isChecked === 'true') {
  // Execute logic for checked state
}
```

- **Data Extraction:**

Extract data stored in custom attributes for JavaScript operations, enabling seamless integration between your HTML and JavaScript code.

```
<html>
<div data-price="49.99">Product Item</div>
</html>
<script>
const product = document.querySelector('div');
const price = parseFloat(product.getAttribute('data-price'));
</script>
```

- **Dynamic URLs:**

Retrieve the href attribute value of anchor (<a>) elements to adapt links dynamically, facilitating smooth navigation within single-page applications.

```
const link = document.getElementById('myLink');
const hrefValue = link.getAttribute('href');
window.location.href = hrefValue;
```

In conclusion, the **getAttribute()** method serves as a bridge between HTML attributes and JavaScript, facilitating seamless communication and data extraction. By embracing its capabilities, you can create dynamic, data-driven, and user-friendly web applications that respond intelligently to user inputs and changing conditions.